COMP5318 Assignment 2: Image Classification

SID1 550378183 SID3

1 Introduction

2 Data

2.1 Data Description and Exploration

2.2 Pre-processing

2.2.1 MLP Feature Standardisation

The MLP used the shared 357-dimensional colour and spatial feature vectors. The shared stratified split provided 25,600 fitting examples and 6,400 validation examples. We standardised each feature using the mean and standard deviation of the fitting subset, then applied the same transformation to validation data. This placed features on comparable scales for gradient-based training without using validation information to fit the scaler.

After model selection, the scaler was refitted on all 32,000 training examples and used to transform the 8,000 test examples. Missing-value imputation was unnecessary because the feature arrays contained no missing or non-finite values.

3 Methods

3.1 Theory

3.1.1 Fully Connected Neural Network

A multilayer perceptron (MLP) combines fully connected layers with nonlinear activation functions to map input features to class predictions. Each layer applies trainable weights and biases, followed by an activation function. We used ReLU, $f(z) = \max(0, z)$, in the hidden layers and softmax in the output layer to produce probabilities for the nine classes. The predicted class was the one with the highest probability.

Training minimised sparse categorical cross-entropy using integer class labels. Backpropagation computed the gradients, and Adam updated the weights in mini-batches. We also tested dropout after each hidden layer. By randomly setting a proportion of activations to zero during training, dropout reduces reliance on particular hidden units and can help limit overfitting.

Using the same extracted features for MLP and Random Forest allowed us to compare their learning methods under a common input representation. The comparison with CNN also involved a difference in representation, since CNN used image pixels rather than these feature vectors.

3.2 Strengths and Weaknesses

3.2.1 MLP in Comparison with Random Forest and CNN

The MLP can learn nonlinear combinations of colour and regional features that may help distinguish the image classes. Using 357 features instead of 2,352 pixel values also reduces the number of first-layer weights compared with an equally wide MLP trained on flattened images. However, wider hidden layers increase model capacity and may overfit, which motivated our tests of dropout and early stopping.

Unlike Random Forest, whose threshold-based splits generally do not require standardisation, MLP training depends on feature scaling, learning rate, and initialisation. Both methods capture nonlinear relationships. The MLP is difficult to interpret, while Random Forest offers feature-importance summaries, although these do not fully explain individual predictions.

The main limitation for image classification is that the extracted features retain only coarse spatial information. CNN filters can learn local texture and spatial patterns directly from images, so we expected CNN to benefit where these details distinguish classes. Whether this leads to better test performance must be checked experimentally.

3.3 Architecture and Hyperparameters

3.3.1 MLP Architecture and Search Design

We used 357 input features, two ReLU hidden layers, and a nine-unit softmax output layer. Two hidden layers allowed successive nonlinear transformations without expanding the search to many network depths. Table 1 lists the three hyperparameters varied: hidden-layer widths, dropout rate, and Adam learning rate. Activation functions, optimiser type, and batch size were fixed, and early stopping was used in every trial.

Table 1: MLP hyperparameter search space and design rationale. Hidden-layer widths are listed in input-to-output order.

Hyperparameter	Candidate values	Rationale
Hidden-layer widths	$(128, 64)$; $(256, 128)$	Compare modelling capacity and computational cost, and assess whether a wider network improves validation performance.
Dropout rate	0.0; 0.2	Test whether moderate dropout improves generalisation relative to no dropout.
Learning rate	0.001; 0.0003	Compare optimisation behaviour and the performance–runtime trade-off under different update sizes.

Grid search evaluated all eight combinations on the same fitting and validation subsets. This small grid allowed us to compare all combinations within the available computational budget. Each model was trained from scratch for up to 60 epochs with a batch size of 128, resetting the random seed to 42 before each trial. Early stopping monitored validation cross-entropy with a patience of eight epochs and restored the weights with the lowest validation loss.

We ranked configurations by validation macro F1, using accuracy to break ties. Macro F1 gave each class equal weight. Thus, validation loss selected the checkpoint within each run, while macro F1 selected the configuration across runs.

The selected configuration was then trained from scratch on all 32,000 training examples, using the epoch count chosen during validation. The test set was reserved for final evaluation.

4 Results and Discussion

4.1 Hyperparameter Tuning

4.1.1 MLP Tuning Results

Table 2 reports the results of all eight configurations on the shared validation set. The selected configuration used hidden-layer widths of $(256, 128)$, a dropout rate of 0.2, and a learning rate of 0.0003. It achieved the highest validation macro F1 of 0.8606 and the highest validation accuracy of 0.8661. Its lowest validation loss occurred at epoch 33; training stopped after epoch 41.

Table 2: Validation results for all eight MLP configurations. Bold values identify the selected configuration. Best epoch is determined by minimum validation loss; epochs run includes the early-stopping patience period. Time includes model construction and fitting, including validation during training, but excludes feature scaling and post-training prediction.

Trial	Hidden widths	Dropout	Learning rate	Best epoch	Epochs run	Val. accuracy	Val. macro F1	Time (s)
1	$(128, 64)$	0.0	0.0010	7	15	0.8344	0.8311	2.40
2	$(128, 64)$	0.0	0.0003	21	29	0.8419	0.8380	4.37
3	$(256, 128)$	0.0	0.0010	8	16	0.8348	0.8322	2.93
4	$(256, 128)$	0.0	0.0003	13	21	0.8455	0.8407	3.86
5	$(128, 64)$	0.2	0.0010	21	29	0.8598	0.8547	4.46
6	$(128, 64)$	0.2	0.0003	50	58	0.8620	0.8570	8.47
7	$(256, 128)$	0.2	0.0010	13	21	0.8622	0.8577	4.07
8	$(\mathbf{256, 128})$	**0.2**	**0.0003**	**33**	**41**	**0.8661**	**0.8606**	**7.61**

Figure 1 visualises the matched comparisons in Table 2. Each pair of bars compares the two dropout rates while keeping the hidden-layer widths and learning rate fixed.

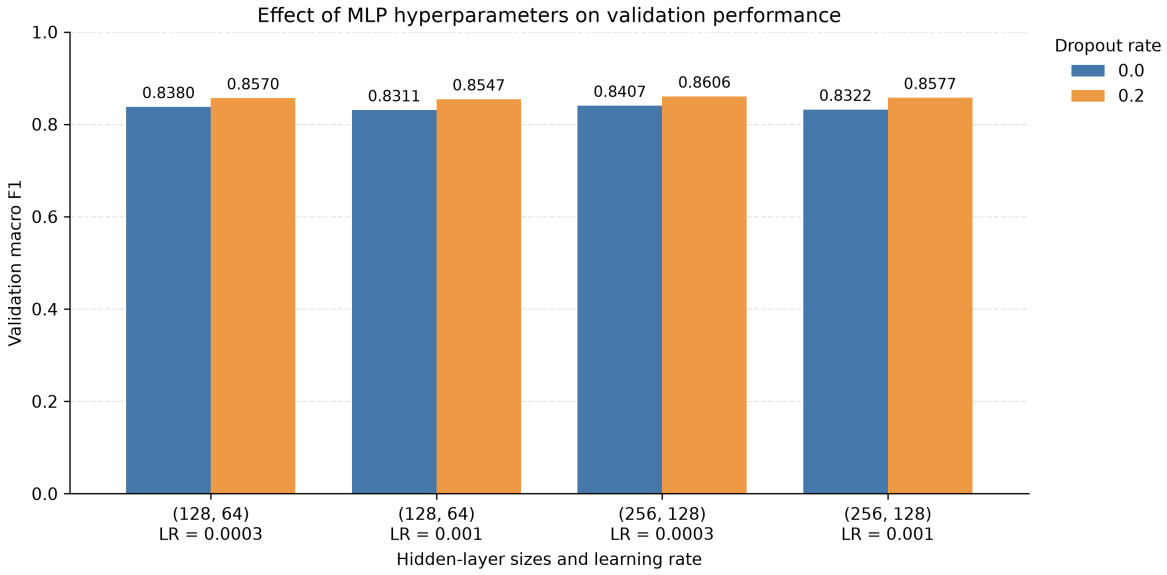


Figure 1: Validation macro F1 across the eight MLP configurations. Each group represents one combination of hidden-layer widths and learning rate; colours distinguish dropout rates of 0.0 and 0.2. Values above the bars show macro F1 to four decimal places.

Effect of dropout. Figure 1 shows that dropout of 0.2 improved validation macro F1 in all four matched comparisons, with gains of 0.0190–0.0255. For example, the wider network at a learning rate of 0.001 improved from 0.8322 to 0.8577. This was the largest effect among the settings tested and supports the use of regularisation for this model. It does not show that 0.2 is the best dropout rate beyond the two values tested.

Effect of network width. Increasing the hidden-layer widths from $(128, 64)$ to $(256, 128)$ improved macro F1 by only 0.0011–0.0036. Although the wider model performed better in all four comparisons, increasing capacity had a smaller effect than adding dropout.

Effect of learning rate and runtime. The lower learning rate improved macro F1 in all four comparisons but required more epochs and longer training. For the wider model with dropout, reducing the rate from 0.001 to 0.0003 raised macro F1 from 0.8577 to 0.8606 and runtime from 4.07 to 7.61 seconds. The smaller update steps may have helped optimisation, but the gain was modest relative to the extra time. We selected this configuration because validation macro F1 was our primary criterion.

These trends were consistent with our expectations for regularisation and smaller learning steps. However, one seed and one validation split are insufficient to establish whether the small differences would persist across repeated runs.

Learning dynamics and early stopping. In Figure 2, both losses fell rapidly at first. Later, training loss kept decreasing while validation loss stayed near 0.40. Validation accuracy also levelled off as training accuracy continued to rise. This suggests that further training mainly improved the fit to the training data rather than validation performance.

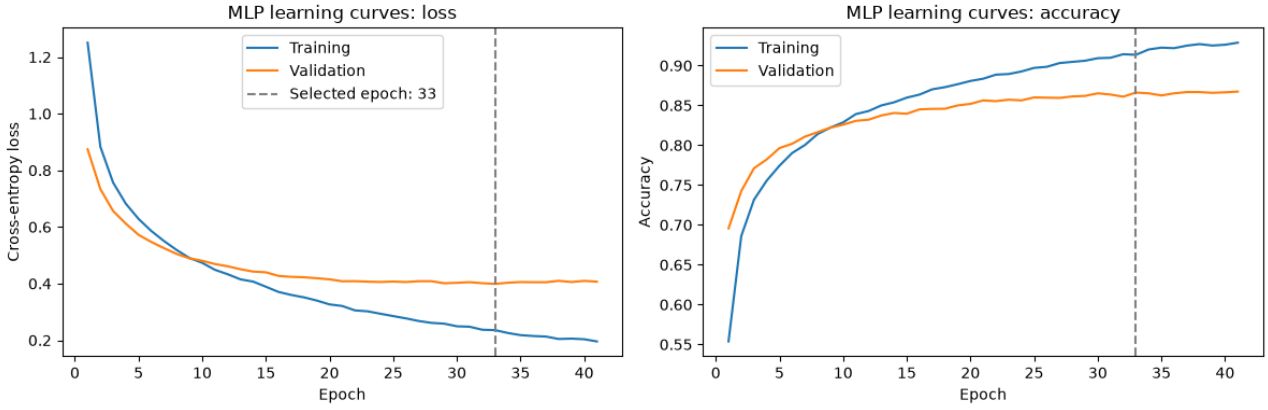


Figure 2: Training and validation learning curves for the selected MLP configuration: hidden layers of 256 and 128 units, dropout rate 0.2, and learning rate 0.0003. The dashed line marks epoch 33, which achieved the lowest validation loss.

Validation loss was lowest at epoch 33, and training stopped at epoch 41 after eight epochs without improvement. The epoch-33 weights were restored, and 33 epochs were subsequently used for final retraining.

The initial advantage of validation over training may partly reflect dropout being disabled during validation and training metrics being collected while the weights were being updated. The curves therefore reflect different evaluation conditions as well as differences between the data subsets.

4.2 Final Model Comparison

4.3 Analysis of Model Performance

4.3.1 MLP Test Performance and Error Analysis

After retraining on all 32,000 training examples, the MLP achieved accuracy of 0.8772, macro F1 of 0.8737, and weighted F1 of 0.8770 on the 8,000 test examples. Test macro F1 was higher than validation macro F1 of 0.8606, but the evaluation subsets and training sizes differed, so this increase cannot be attributed to retraining alone.

Variation across classes. Classes 0, 1, and 3 achieved F1 scores of approximately 0.96 or higher, while classes 2, 6, and 7 scored below 0.80. Class 7 had the lowest F1 of 0.7543, with precision of 0.7337 and recall of 0.7761. Class 2 also had low recall of 0.7412, meaning that roughly one quarter of its examples were misclassified. These differences show that the similar macro and weighted F1 scores did not imply equally strong performance across classes.

Patterns of misclassification. Figure 3 shows the error counts and row-normalised proportions. The latter allows comparison across classes with different test-set sizes.

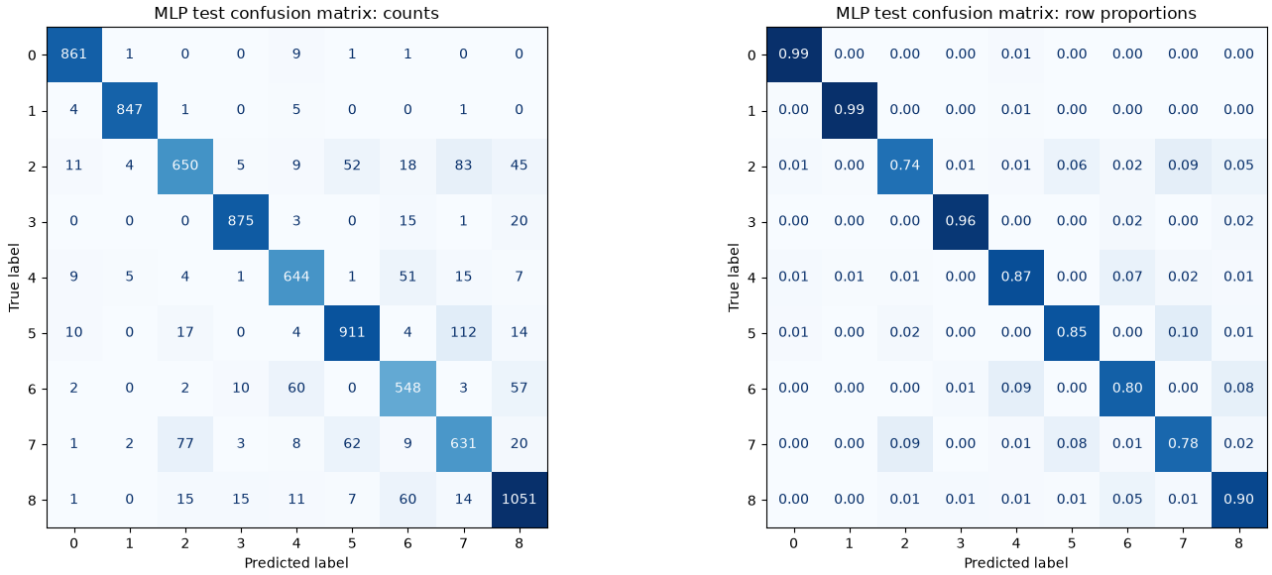


Figure 3: Confusion matrices for the final MLP on 8,000 test examples. Left: numbers of predictions. Right: proportions normalised within each true class, with diagonal entries corresponding to class recall.

The largest error count was from class 5 to class 7: 112 examples, or 10.45% of class 5. Another 62 errors occurred in the reverse direction. Classes 2 and 7 were also confused with each other, with 83 and 77 errors, respectively. Predictions from classes 5 and 2 into class 7 help explain its low precision. Class 6 was frequently assigned to classes 4 and 8, with 60 and 57 errors.

These errors suggest that the model struggled to separate particular class pairs. The colour and coarse spatial features may omit useful detail, but identifying the visual cause would require inspecting the misclassified images. The confusion matrix indicates where to focus that inspection rather than explaining the cause by itself.

Computational cost. Final standardisation, model construction, and training took 6.44 seconds; test transformation and prediction took 0.09 seconds. These timings exclude feature extraction, file loading, metric calculation, and hyperparameter search. The MLP was therefore quick to fit and evaluate on precomputed features in our environment. A fair comparison with RF and CNN requires consistent hardware and timing boundaries.

5 Conclusion

MLP findings. The MLP achieved test accuracy of 0.8772 and macro F1 of 0.8737, with final standardisation and training taking 6.44 seconds. Within our grid, dropout improved validation performance more than increasing network width.

The model offers a computationally inexpensive option when the shared features are available, although performance remained weaker for several classes.

Limitations. We tested eight configurations using one split and one seed, so the stability of their ranking is unknown. The engineered features may also discard useful image detail, and the reported runtime excludes their extraction.

Future work. Repeated validation with several seeds would allow us to report mean performance and variability. A broader search could then refine the promising settings. Inspecting confused class pairs could guide feature improvements, while timing the complete pipeline on common hardware would improve the runtime comparison. Further tuning should use validation data, keeping the test set reserved for evaluation.

6 Reflection

Member B: 550378183

The main lesson I learned was to judge model complexity using validation results rather than training performance. Increasing network width gave only small gains, whereas dropout had a larger effect. The learning curves also showed that training loss could keep falling without a comparable improvement on validation data. This made the purpose of regularisation and early stopping clearer to me.

Examining individual classes changed how I interpreted the overall result. Accuracy of 0.8772 looked encouraging, but class 7 had an F1 score of only 0.7543. In future experiments, I would examine class-level errors alongside aggregate scores and repeat promising settings across seeds before treating a small improvement as reliable.

7 AI Acknowledgement